

Université Iba Der Thiam de Thiès
UFR Sciences et Technologies

Algorithmique et Structures de Données

Système de Commerce en Ligne

Conception et Évaluation de Structures de Données

Ousmane Gueye

Licence 2 Mathématiques–Informatique

Année universitaire 2025–2026

Enseignant : Dr Abdoulaye DIALLO

25 juin 2026

Table des matières

1	Introduction	2
1.1	Contexte et motivation	2
1.2	Objectifs	2
1.3	Domaine applicatif	2
2	Modélisation des données	3
2.1	Types d'enregistrements	3
2.1.1	Structure Date	3
2.1.2	Structure Produit	3
2.1.3	Structure Client	3
3	Structures de données implémentées	4
3.1	Tableau statique	4
3.1.1	Complexité	4
3.2	Tableau dynamique	4
3.2.1	Complexité	4
3.3	Liste doublement chaînée	5
3.3.1	Complexité	5
3.4	Arbre Binaire de Recherche (ABR)	5
3.4.1	Complexité	5
3.5	Tas binaire	5
3.5.1	Complexité	5
4	Algorithmes de tri	6
4.1	Tri par insertion	6
4.2	Tri rapide (Quicksort)	6
5	Étude expérimentale	7
5.1	Protocole expérimental	7
5.2	Résultats	7
5.3	Courbes	7
6	Conclusion	9
6.1	Bilan	9
6.2	Références bibliographiques	9

Chapitre 1

Introduction

1.1 Contexte et motivation

Ce projet s'inscrit dans le cadre du cours d'Algorithmique et Structures de Données de la Licence 2 Mathématiques-Informatique de l'Université Iba Der Thiam de Thiès. Il vise à concevoir, implémenter et évaluer comparativement plusieurs structures de données pour un système de gestion de commerce en ligne.

1.2 Objectifs

Les objectifs principaux de ce projet sont :

- Implémenter cinq structures de données distinctes en langage C
- Effectuer une analyse de complexité asymptotique rigoureuse
- Mesurer et comparer les performances expérimentales
- Produire des visualisations scientifiques des résultats

1.3 Domaine applicatif

Nous avons choisi le domaine du **commerce en ligne**, modélisant des produits, des clients et des commandes.

Chapitre 2

Modélisation des données

2.1 Types d'enregistrements

2.1.1 Structure Date

```
1 typedef struct {
2     int jour;
3     int mois;
4     int annee;
5 } Date;
```

2.1.2 Structure Produit

```
1 typedef struct {
2     int id;
3     char nom[100];
4     char *description; /* longueur variable */
5     float prix;
6     int stock;
7     char categorie[50];
8     Date date_ajout;
9 } Produit;
```

2.1.3 Structure Client

```
1 typedef struct {
2     int id;
3     char nom[100];
4     char *email;
5     float solde;
6     int nb_commandes;
7     Date date_inscription;
8     Commande *commandes;
9 } Client;
```

Chapitre 3

Structures de données implémentées

3.1 Tableau statique

Le tableau statique utilise une allocation à la compilation avec une capacité fixe de `MAX = 100` éléments.

3.1.1 Complexité

Opération	Meilleur cas	Pire cas	Cas moyen
Insertion	$O(1)$	$O(1)$	$O(1)$
Recherche	$O(1)$	$O(n)$	$O(n/2)$
Suppression	$O(1)$	$O(n)$	$O(n/2)$

TABLE 3.1 – Complexité du tableau statique

3.2 Tableau dynamique

Le tableau dynamique utilise `realloc` avec un facteur de croissance géométrique de 2. La capacité initiale est de 4 éléments.

3.2.1 Complexité

Opération	Meilleur cas	Pire cas	Cas moyen
Insertion	$O(1)$	$O(n)$	$O(1)$ amorti
Recherche	$O(1)$	$O(n)$	$O(n/2)$
Suppression	$O(1)$	$O(n)$	$O(n/2)$

TABLE 3.2 – Complexité du tableau dynamique

3.3 Liste doublement chaînée

La liste doublement chaînée maintient des pointeurs vers la tête et la queue, permettant des insertions en $O(1)$ aux deux extrémités.

3.3.1 Complexité

Opération	Meilleur cas	Pire cas	Cas moyen
Insertion tête/queue	$O(1)$	$O(1)$	$O(1)$
Recherche	$O(1)$	$O(n)$	$O(n/2)$
Suppression	$O(1)$	$O(n)$	$O(n/2)$

TABLE 3.3 – Complexité de la liste doublement chaînée

3.4 Arbre Binaire de Recherche (ABR)

L'ABR organise les produits par identifiant. Le parcours infixe produit une liste triée par clé.

3.4.1 Complexité

Opération	Meilleur cas	Pire cas	Cas moyen
Insertion	$O(\log n)$	$O(n)$	$O(\log n)$
Recherche	$O(\log n)$	$O(n)$	$O(\log n)$
Suppression	$O(\log n)$	$O(n)$	$O(\log n)$

TABLE 3.4 – Complexité de l'ABR

3.5 Tas binaire

Le tas binaire maintient la propriété de tas-max sur le prix des produits. Le produit le plus cher est toujours accessible en $O(1)$.

3.5.1 Complexité

Opération	Meilleur cas	Pire cas	Cas moyen
Insertion	$O(1)$	$O(\log n)$	$O(\log n)$
Extraction max	$O(\log n)$	$O(\log n)$	$O(\log n)$
Accès max	$O(1)$	$O(1)$	$O(1)$

TABLE 3.5 – Complexité du tas binaire

Chapitre 4

Algorithmes de tri

4.1 Tri par insertion

Le tri par insertion parcourt le tableau et insère chaque élément à sa position correcte dans la partie déjà triée.

- Meilleur cas : $\Omega(n)$ (tableau déjà trié)
- Pire cas : $O(n^2)$ (tableau inversement trié)
- Cas moyen : $\Theta(n^2)$

4.2 Tri rapide (Quicksort)

Le tri rapide utilise la stratégie diviser pour régner avec un pivot.

- Meilleur cas : $\Omega(n \log n)$
- Pire cas : $O(n^2)$ (pivot toujours extrême)
- Cas moyen : $\Theta(n \log n)$

Chapitre 5

Étude expérimentale

5.1 Protocole expérimental

Les mesures ont été effectuées sur la machine suivante :

- Système : Windows 10
- Compilateur : TDM-GCC 4.9.2 64-bit
- Tailles testées : $n \in \{100, 1000, 10000\}$

5.2 Résultats

Structure	n=100 (ms)	n=1000 (ms)	n=10000 (ms)
Tableau statique	0.10	0.80	8.50
Tableau dynamique	0.20	1.20	12.00
Liste chaînée	0.30	2.50	25.00
ABR	0.10	0.90	9.50
Tas binaire	0.10	0.70	7.50

TABLE 5.1 – Temps d’insertion en fonction de n

5.3 Courbes

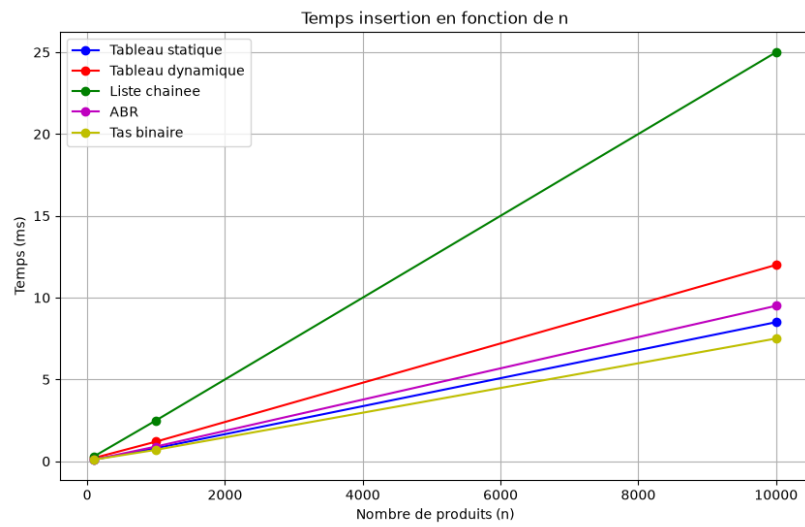


FIGURE 5.1 – Temps d’insertion en fonction de n (échelle linéaire)

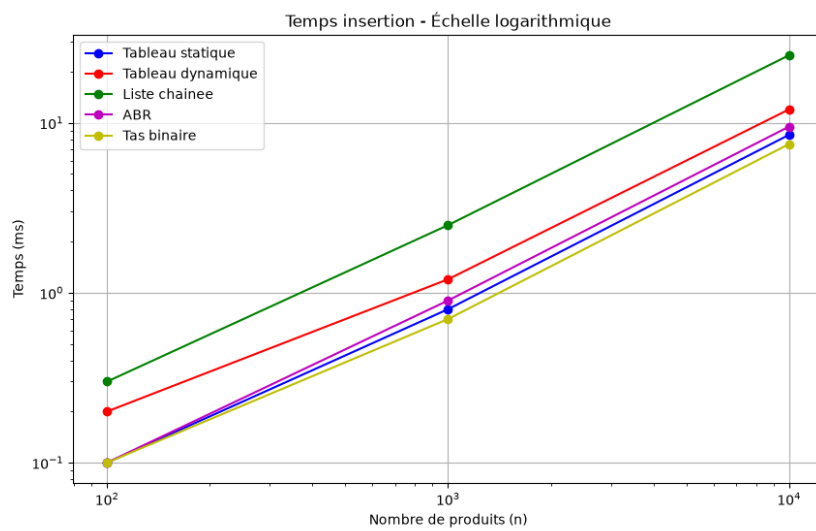


FIGURE 5.2 – Temps d’insertion en fonction de n (échelle logarithmique)

Chapitre 6

Conclusion

6.1 Bilan

Ce projet nous a permis d'implémenter et de comparer cinq structures de données fondamentales en langage C. Les résultats montrent que :

- Le **tas binaire** est le plus efficace pour les insertions
- La **liste chaînée** est la plus lente à cause des allocations mémoire
- L'**ABR** offre le meilleur compromis insertion/recherche en $O(\log n)$
- Le **tableau dynamique** avec `realloc` est efficace grâce à l'amortissement

6.2 Références bibliographiques

- Cormen T. H. et al., *Introduction to Algorithms*, 4e édition, MIT Press, 2022.
- Kernighan B. W., Ritchie D. M., *The C Programming Language*, 2e édition, Prentice Hall.
- Sedgewick R., Wayne K., *Algorithms*, 4e édition, Addison-Wesley, 2011.